Continuum Categories

Contract Categories, Realization Profunctors, and Witness Squares

2026-04-09

1 Context

These notes record a minimal categorical story for the Continuum stack. The target is not a maximal formalism. The target is the smallest defensible account that explains:

1. what Wesley compiles,
2. what the host runtimes consume,
3. why the right semantic shape is profunctorial,
4. how witness output upgrades generation into conformance.

The intended runtime family is the present Continuum surface:

- Echo as a Continuum participant,
- GitWarp as a Continuum participant,
- TTD as the host-neutral debugger protocol surface,
- Fuse-style materializations as host adapters over worldlines, strands, or braids.

Continuum is the protocol for exchanging witnessed causal history. It is not a runtime hierarchy, and it does not make Echo the owner of GitWarp's participant role.

2 Design Claim

Principle 1 (Continuum split)**.** The authored contract surface, the host runtime presentation surface, and the conformance witness surface are distinct layers. Treating any one of them as the others produces shadow authority and contract drift.

Observation 1 (Wesley is not one profunctor)**.** Wesley should not be modeled as a single profunctor. Wesley is better modeled as a compiler that, for each host runtime, produces a realization profunctor, a canonical compiled presentation, and a local witness that the presentation realizes the authored contract.

3 The Contract Category

Definition 1 (Authored contract category)**.** Let $\mathcal{C}$ be the category of authored Continuum contracts.

Objects of $\mathcal{C}$ are admitted semantic contract families: protocol schemas, observer envelopes, receipt families, coordinate families, rewrite declarations, footprint declarations, and related shared noun families.

A morphism

$$\alpha : c \to c'$$

in $\mathcal{C}$ is a *refinement embedding* from a weaker contract c into a stronger contract c'. Concretely, c' preserves the obligations of c and adds further structure, fields, laws, or distinctions.

This direction is deliberate. If c' refines c, then any runtime surface that realizes c' should also realize c after forgetting the extra structure. The variance of the realization profunctor below captures exactly that rule.

Observation 2 (SDL is syntax, not the category)**.** GraphQL SDL is a concrete authored syntax for presenting objects of $\mathcal{C}$. It is not itself the semantic category. The category lives at the level of admitted contract meaning, not raw source files.

4 Host Presentation Categories

Definition 2 (Host presentation category)**.** For each host $H \in \mathcal{H}$, let $\mathcal{R}_H$ be the category of runtime presentations for that host.

Objects of $\mathcal{R}_H$ are host-facing generated or stabilized surfaces: TypeScript types, Rust types, manifests, codecs, protocol envelopes, artifact bundles, adapter stubs, and other host-native realizations.

A morphism

$$\beta : r \to r'$$

in $\mathcal{R}_H$ is a lawful host-side adaptation that preserves the semantic meaning required by the contract family under discussion.

Examples of hosts include:

$$H \in \{\mathsf{Echo},\ \mathsf{GitWarp},\ \mathsf{TTD},\ \mathsf{Fuse}\}.$$

Observation 3 (Host categories do not own contract truth)**.** The host presentation categories carry realized surfaces, not authored authority. Runtime convenience, storage shape, and protocol framing belong in the host category only after the authored contract boundary has been named.

5 Realization Profunctors

Definition 3 (Realization profunctor)**.** For each host H, define a profunctor

$$\mathsf{Real}_H : \mathcal{C}^{op} \times \mathcal{R}_H \to \mathbf{Set}$$

where $\mathsf{Real}_H(c, r)$ is the set of witnesses that runtime presentation r faithfully realizes contract c for host H.

The two variances carry the core Continuum rules:

- **Contravariant in the contract.** If $\alpha : c \to c'$ is a refinement embedding, then a witness for the stronger contract c' induces a witness for the weaker contract c.
- **Covariant in the runtime presentation.** If $\beta : r \to r'$ is a lawful host adaptation, then a witness for r transports to a witness for r'.

So for every $\alpha : c \to c'$ in $\mathcal{C}$ and $\beta : r \to r'$ in $\mathcal{R}_H$, the profunctor yields a transport map

$$\mathsf{Real}_H(\alpha, \beta) : \mathsf{Real}_H(c', r) \to \mathsf{Real}_H(c, r').$$

Observation 4 (Why the profunctor shape is the right one)**.** The contract side and the runtime side do not vary in the same direction. Strengthening a contract moves opposite to realization transport, while lawful runtime adaptation moves with it. That asymmetry is exactly what makes the realization story profunctorial rather than functorial.

6 Canonical Compilation

Definition 4 (Host-indexed compile path)**.** For each host H, Wesley provides a contravariant compile path

$$\mathsf{Compile}_H : \mathcal{C}_{\mathrm{adm}}^{op} \to \mathcal{R}_H$$

defined on the admissible contract families carried by that compile path.

Objectwise, $\mathsf{Compile}_H(c)$ is the canonical compiled presentation of contract c for host H. On a refinement embedding $\alpha : c \to c'$, the arrow

$$\mathsf{Compile}_H(\alpha) : \mathsf{Compile}_H(c') \to \mathsf{Compile}_H(c)$$

is the host-side forgetting or adaptation map from the richer compiled surface to the weaker one.

Definition 5 (Witness assignment)**.** For each admissible contract c, Wesley emits a local witness

$$\mathsf{wit}_H(c) \in \mathsf{Real}_H\big(c, \mathsf{Compile}_H(c)\big).$$

Proposition 1 (Witness-square law)**.** For each refinement embedding $\alpha : c \to c'$ in the admissible contract surface, the compiled surfaces and witnesses satisfy

$$\mathsf{Real}_H\big(\alpha, \mathsf{Compile}_H(\alpha)\big)\big(\mathsf{wit}_H(c')\big) = \mathsf{wit}_H(c).$$

This is the core conformance condition. It says that the witness for the stronger compiled contract, transported across the contract refinement and the runtime adaptation, agrees with the witness for the weaker compiled contract.

Observation 5 (Why witness output matters)**.** Generation without the witness-square law is only code emission. The witness surface is what upgrades the compile path into a conformance claim.

7 Publication Boundaries

The categorical story also clarifies Wesley's publication-boundary role.

Principle 2 (Publication-boundary rule)**.** For an admitted shared noun family, the repo must name:

1. the authored home carrying the contract object in $\mathcal{C}$,
2. the host-indexed compile path $\mathsf{Compile}_H$ that consumes it,
3. the stable generated surface $\mathsf{Compile}_H(c)$ expected by consumers,
4. the local witness $\mathsf{wit}_H(c)$ or equivalent conformance output.

If any of these are missing, the system has target-state intent but not a fully admitted contract lane.

8 Observer Geometry and Optics

The present account also separates three ideas that should not be collapsed.

1. **Observer Geometry** studies the geometry of projection: observers, apertures, degeneracies, translation costs, mediator paths, and observer-relative logical preservation.
2. **WARP optics** studies the full lawful rewrite envelope around a projection. In the current working notation, a WARP optic is a tuple

$$\Omega = (\pi, \mathcal{F}, \rho, \omega, \sigma)$$

   consisting of projection, focus boundary, local rewrite, witness, and lawful reintegration.
3. **Wesley** is the contract compiler that turns authored shared surfaces into host-indexed runtime presentations and witness squares. Wesley does not own the runtime semantics of observers or optics; it preserves their authored contract boundary and realizes them coherently across hosts.

Observation 6 (Projection versus full optic)**.** Observer Geometry is primarily about the geometry of π. Optics studies the larger object in which π lives. Wesley compiles authored contracts that may name either projection-only surfaces or richer optic-bearing surfaces, but it should not be identified with the optic itself.

9 Host-Indexed Optic Interpretations

The profunctor account becomes especially useful once authored contracts carry rewrite and footprint structure.

An authored optic-bearing contract may then be interpreted across hosts by different realization contexts:

Host or realization context	Interpreted role
Echo	execute Echo-local rewrites
GitWarp	admit and materialize causal patches
TTD	expose debugger playback and explanation surfaces
Fuse	materialize editable filesystem projections
footprint analysis	derive read/write/delete scope
witness extraction	derive local reversibility residue

The important point is not that these are identical implementations. The important point is that they are host-indexed realizations of one authored contract family rather than handwritten shadows with approximately matching names.

10 Judgment Surfaces Are Downstream

Wesley's judgment-bridge role can also be expressed categorically, but it is a downstream stage.

For each host H, one may define a category $\mathcal{J}_H$ of operator-facing judgment surfaces and a functor

$$\mathsf{Judge}_H : \mathcal{R}_H \to \mathcal{J}_H$$

that turns substrate-facing realized surfaces into operator-facing statuses, risk classes, confidence adjustments, explanations, or gate results.

Observation 7 (Judgment does not replace realization)**.** Judgment surfaces do not substitute for contract realization witnesses. They consume realized runtime facts. They do not retroactively become the fact layer.

11 What This Story Does Not Yet Claim

- It does not claim a complete bicategory or equipment of Continuum contracts, observers, and optics.
- It does not yet define full optic laws for authored rewrites.
- It does not identify a canonical enrichment for observer distance, aperture, or budgeted translation cost.
- It does not prove that every practical Wesley compile path already satisfies the witness-square law; that remains a repo-by-repo burden of evidence.

12 Future Lift

If the stack matures further, the natural next lift is toward a double categorical picture:

- objects: contract domains or admitted noun families,
- vertical arrows: structure-preserving refinements,
- horizontal arrows: realization relations, translators, or observer relations,
- squares: witness-bearing coherence between refinement and realization.

But the present note does not need that extra machinery to be useful. The realization profunctor plus witness-square law already explains why Wesley's role is stronger than plain code generation and narrower than substrate ownership.

13 One-Sentence Summary

Continuum is best modeled by a contract category, host-indexed runtime presentation categories, realization profunctors between them, and local witness squares showing that Wesley's compiled surfaces really realize the authored contracts they claim to carry.